Giugno 2025

1 Cryptography

Consider the following cryptographic algorithm based on the idea of substituting each letter of the alphabet m with $(am + b)$ mod 26, where the key is given by k = $\langle a, b\rangle$ with a and b being positive integers in the interval [0,25].

Show, using an example, that if a and 26 are not relatively prime, then the schema is not usable.

Perché un sistema di cifratura sia "usabile", deve essere **reversibile** (decifrabile). Se cifro "CIAO", devo poter tornare esattamente a "CIAO".

Per poter tornare indietro matematicamente, il numero a deve avere un "inverso moltiplicativo". Questo accade solo se il Massimo Comun Divisore (MCD) tra a e 26 è 1. In altre parole, a e 26 devono essere **coprimi** (non devono avere divisori in comune a parte 1).

L'esercizio chiede di mostrare cosa succede se **NON** rispettiamo questa regola.

- **Scelta della chiave "sbagliata":**

Viene scelto $a = 13$ e $b = 0$.

Il problema è evidente: 13 e 26 non sono coprimi, perché 26 è divisibile per 13 $(26 = 13 \cdot 2)$. Quindi $\text{MCD}(13, 26) = 13 \neq 1$.

Hai cifrato due lettere diverse ('c' ed 'e') e hai ottenuto **lo stesso risultato** ('a'). Questo fenomeno si chiama **collisione**.

La funzione, come dice il testo, **non è iniettiva**.
Se chi riceve il messaggio legge una 'A' (o il numero 0), non ha alcun modo di sapere se l'originale era una 'C', una 'E' (o anche una 'A', 'O', ecc.). Poiché ci sono più possibilità e non si può distinguere quale sia quella giusta, **la decifratura è impossibile**.

In sintesi

Il sistema è inutilizzabile perché scegliendo un valore a che condivide fattori con 26 (come 13 o 2), perdi informazioni durante la moltiplicazione, facendo "collassare" lettere diverse sullo stesso risultato cifrato.

Risposta:

Si consideri $a = 13$ e $b = 0$.
Si nota subito che, essendo il MCD(13,26) $= 13 \neq 1$, a e 26 non sono coprimi.
Se si considerano due lettere m_1 e m_2 diverse tra di loro, per esempio $m_1 = c = 2$ e $m_2 = e = 4$, allora l'algoritmo non funziona:

$$(13 \cdot 2 + 0) = 26 \bmod 26 = 0$$
$$(13 \cdot 4 + 0) = 52 \bmod 26 = 0$$

sia m_1 che m_2 vengono mappate in 0, ossia a, rendendo impossibile il recupero di m_1 e m_2. In questo caso la funzione matematica non è iniettiva, proprietà matematica fondamentale per gli algoritmi di cifratura e decifratura.

- **Il test (La collisione):**

Vengono prese due lettere diverse come input per vedere se producono risultati diversi.

- Lettera 1: c (che corrisponde al numero 2).
- Lettera 2: e (che corrisponde al numero 4).

- Applichiamo la formula:

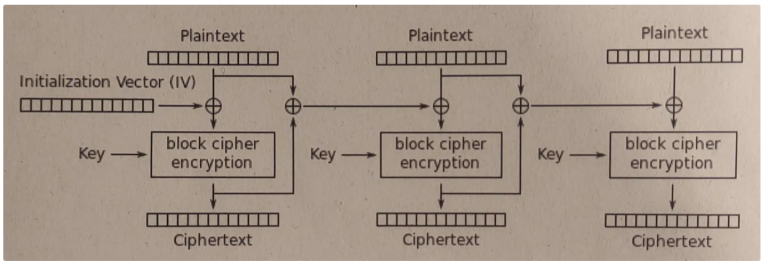
1. Per $m_1 = 2$:

1. (che corrisponde alla lettera 'A')
2. Per $m_2 = 4$:

2. (perché 52 è esattamente $26 \cdot 2$, quindi il resto è 0. Corrisponde ancora alla lettera 'A').

Febbraio 2024

2. Consider the following mode of operation used for encryption:



$c_0 = H(x_0) = E(k, x_0 \oplus IV)$
$c_i = H(x_i) = E(k, x_i \oplus (c_{i-1} \oplus x_{i-1}))$

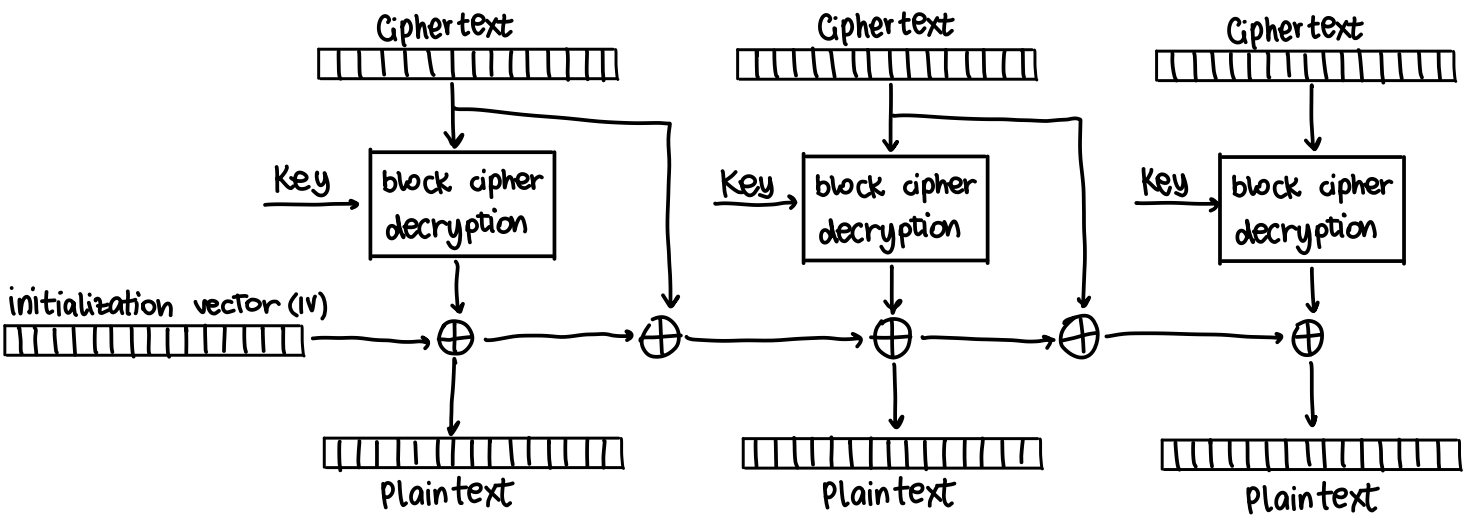
Draw the corresponding schema to be used for decryption.

▼ Suggerimento in caso di difficoltà

Per risolvere questi esercizi dove viene chiesto di fornire lo schema di cifratura o decifratura, basta immaginare di posare uno specchio alla base dello schema e vederlo, così, ribaltato. L'immagine ribaltata è lo schema inverso che serve per invertire il processo. Attenzione alla direzione delle frecce che cambia anche essa.

$$p_0 = H(x_0) = D(k, x_0) \oplus IV$$
$$p_i = H(x_i) = D(k, x_i) \oplus (p_{i-1} \oplus x_{i-1})$$

Questo schema è una variante del CBC (Cipher Block Chaining)



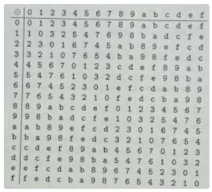
Gennaio 2024

2. Consider a block cipher operating on 4-bit blocks, written as hexadecimal digits. For one particular key K, it implements the following permutation:

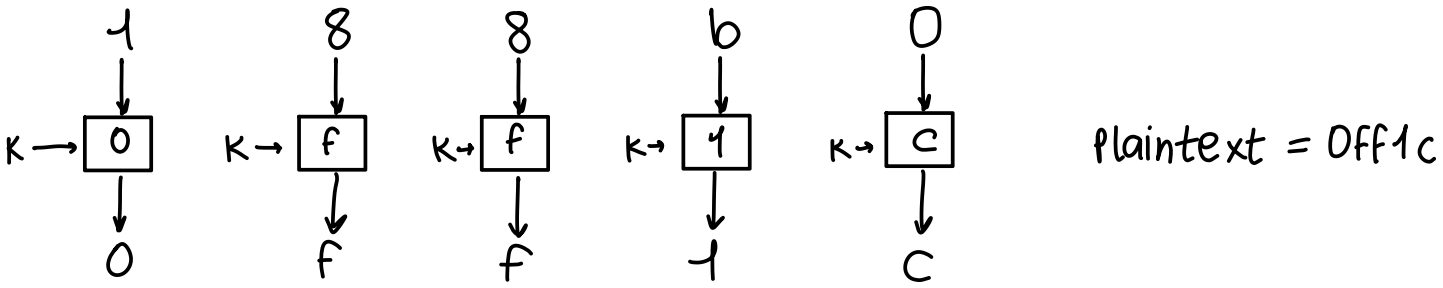
m	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$E_k(m)$	1	b	5	c	7	e	2	a	4	9	f	d	0	3	6	8

Using this key K, decrypt the following three ciphertexts according to the indicated modes of operation. You may find it useful the following table:

Nota: la tabella torna utile per computare le operazioni di XOR con il CBC se non ti scrivi la tabellina con le rappresentazioni binarie come fatto sotto (es. $3 \oplus 1 = 2$, riga 3 colonna 1 il risultato è 2).

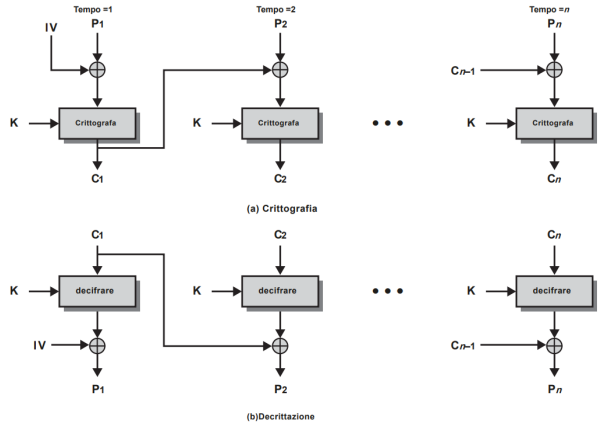


a) ECB (Electronic Code Book) : 188b0

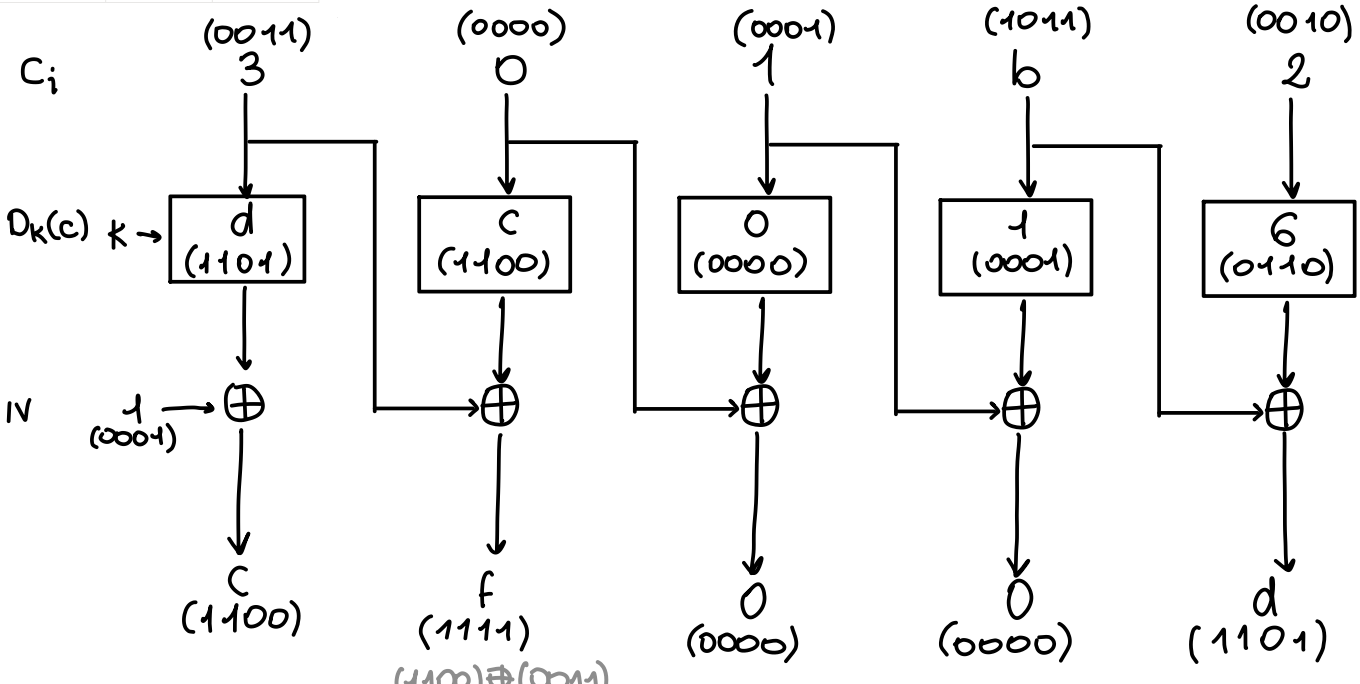


b) CBC (Cipher-Block Chaining) : 301b2 [Assume IV = 1]

Decimale	Esadecimale	Binario
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	a	1010
11	b	1011
12	c	1100
13	d	1101
14	e	1110
15	f	1111



Plaintext = cf00d



Giugno 2023

1. **Do substitution ciphers preserve the frequency of the letters?**
   - [x] TRUE
   - [ ] FALSE
   - ▼ Risposta

     True.

2. **Do transposition ciphers preserve the frequency of the letters?**
   - [x] TRUE
   - [ ] FALSE
   - ▼ Risposta

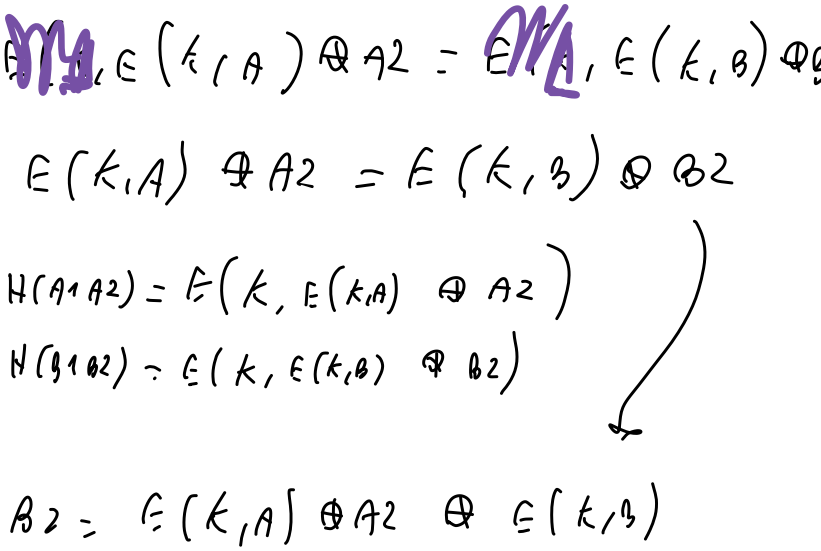
     True.

3. **The number of transposition ciphers applicable to a string of N bits is**
   - ▼ Risposta

     `N!`

4. **The number of substitution ciphers applicable to a string of N bits is**
   - ▼ Risposta

     `2^N!`



Febbraio 2023

1. Consider the following definition of a hash function based on a cryptographic algorithm.

Let K be a given cryptographic key and $M = M_1M_2...M_n$ (with $n \geq 1$) be a message where $M_1....M_n$, are blocks of the same length:

$H(M_1) = E(K, M_1)$

$H(M_1 \dots M_iM_{i+1}) = E(K, H(M_1 \dots M_i) \oplus M_{i+1})$ for i = 1,..., n – 1

The schema is not secure.
Show that given a message A_1A_2 and an arbitrary block B_1 it is possible to determine a block B_2 such that $H(B_1B_2) = H(A_1A_2)$, i.e. H is not weak collision resistant.

- ▼ Soluzione

  $H(B_1, B_2) =? = H(A_1, A_2)$

  $E(K, E(K, B_1) \oplus B_2) = E(K, E(K, A_1) \oplus A_2)$

  $E(K, H(B_1) \oplus B_2) = E(K, H(A_1) \oplus A_2)$ (osservazione: dato che le due cifrature devono essere identiche e la chiave usata è la stessa, allora il contenuto dev'essere lo stesso)

  $H(B_1) \oplus B_2 = H(A_1) \oplus A_2$

  $B_2 = E(K, B_1) \oplus E(K, A_1) \oplus A_2 = (H(A_1) \oplus A_2) \oplus H(B_1)$

1. Il Contesto e la Funzione Hash

L'esercizio definisce una funzione hash H che processa un messaggio M diviso in blocchi (M_1, M_2, \dots).
La regola è ricorsiva:

- **Primo blocco:** L'hash del primo blocco è semplicemente la cifratura del blocco stesso:

  $$H(M_1) = E(K, M_1)$$

- **Blocchi successivi:** L'hash al passo successivo si ottiene cifrando lo XOR ($\oplus$) tra l'hash precedente e il blocco corrente:

  `H(M_1 \dots M_{i+1}) = E(K, H(M_1 \dots M_i) \oplus M_{i+1})`

2. L'Obiettivo (L'Attacco)

L'esercizio chiede di dimostrare che questa funzione **non è resistente alle collisioni deboli** (in inglese second preimage resistant).

In parole semplici: Ti viene dato un messaggio A (composto da A_1, A_2). Devi riuscire a costruire un secondo messaggio falso B (composto da B_1, B_2) che abbia lo **stesso identico hash finale** di A.
Ti viene data la libertà di scegliere B_1 come vuoi, e devi calcolare quale deve essere B_2 per far funzionare l'inganno.

3. La Soluzione (Passaggio per Passaggio)

La parte "Soluzione" nell'immagine svolge l'algebra necessaria per trovare B_2.

Passo 1: Impostare l'uguaglianza
Vogliamo che l'hash finale dei due messaggi sia identico:

$$H(B_1, B_2) = H(A_1, A_2)$$

Passo 2: Espandere la definizione
Applichiamo la formula data dal testo per due blocchi. L'hash di due blocchi è la cifratura dell'hash del primo blocco XOR il secondo blocco.
Quindi l'equazione diventa:

$$E(K, H(B_1) \oplus B_2) = E(K, H(A_1) \oplus A_2)$$

(Nota: nell'immagine sostituiscono anche $H(B_1)$ con $E(K, B_1)$, è la stessa cosa).

Passo 3: Rimuovere la cifratura esterna (Punto Cruciale)
Qui c'è l'osservazione fondamentale scritta tra parentesi nell'immagine.
Poiché E è un algoritmo di cifratura (una permutazione), è invertibile e deterministico. Se $E(K, X) = E(K, Y)$, allora necessariamente $X = Y$.

Possiamo quindi "cancellare" la E(K, \dots) esterna da entrambi i lati:

$$H(B_1) \oplus B_2 = H(A_1) \oplus A_2$$

Passo 4: Isolare l'incognita (B_2)

Ora dobbiamo trovare B_2. Usiamo le proprietà dello XOR. Se facciamo lo XOR di entrambi i lati con $H(B_1)$, otteniamo B_2 da solo (perché $X \oplus X = 0$):

$$B_2 = H(A_1) \oplus A_2 \oplus H(B_1)$$

Passo 5: Sostituire con i valori noti
Sapendo che l'hash di un singolo blocco è $E(K, \text{blocco})$, sostituiamo i termini $H(A_1)$ e $H(B_1)$:

$$B_2 = E(K, A_1) \oplus A_2 \oplus E(K, B_1)$$

Conclusione

Hai dimostrato che il sistema è insicuro.
Se un attaccante conosce il messaggio originale A_1, A_2, può scegliere un blocco arbitrario B_1 (magari un inizio di frase malevolo) e calcolare matematicamente un blocco "spazzatura" B_2 tale che il computer crederà che il messaggio B sia valido quanto A.

Gennaio 2023

1. Proof of work (PoW) is a form of cryptographic proof in which one party (the prover) proves to others (the verifiers) that a certain amount of a specific computational effort has been expended. One popular approach (HashCash) to PoW amounts to the problem of finding a string X whose SHA-1 hash code begins with N binary zeros, where N is a given positive integer. Assume that computing the SHA-1 hash code takes at most 1 ms.

b. How long does it take to solve the HashCash problem in the worst case if N=10?

   a. 1 sec
   b. 2 sec
   c. 1 min
   d. 2 min
   e. None of the above

▼ Soluzione

a. 1 sec - Bisogna indovinare una stringa di 10 bit nel caso peggiore, ossia dopo aver provato tutte le possibili combinazioni, essendo la stringa binaria sono 2^{10} possibili output, computando un codice di hash ogni millisecondo, ci vorranno 2^{10} millisecondi, ossia 1024 millisecondi, ossia all'incirca 1 secondo.

b. What is the probability to solve the HashCash problem in less than 10 sec if N=20?

▼ Risposta

P = 0,01

P("probabilità di azzeccare un codice di hash con un tentativo") = $\frac{1}{2^{20}}$ dove 2^{20} sono tutti i possibili output della funzione di hash.

P("probabilità di azzeccare un codice di hash in 10 secondi") = $10000 \cdot \frac{1}{2^{20}} = \frac{10000}{1048576} = \frac{10^4}{10^6} = \frac{1}{100} = 0,01 = 1\%$ (10 sec = 10000 ms = 10000 tentativi)

c. What is the minimum value of N for which the solution to the HashCash problem takes at least 1 hour?

▼ Risposta

N = 22

1 h = 60 m = 3.600 sec = 3.600.000 ms → in 1 ora si computano 3.600.000 codici hash diversi

P("probabilità di azzeccare un codice di hash con un tentativo") = $\frac{1}{2^N}$

P("probabilità di azzeccare un codice di hash in 1 ora") = $\frac{1}{2^N} \cdot 3.600.000 = \frac{3.600.000}{2^N}$

Si vuole che la soluzione venga trovata dopo ALMENO un'ora, quindi bisogna porre la disugualianza:

$\frac{3.600.000}{2^N} < 1 \rightarrow 2^N > 3.600.000$ (numero di tentativi diviso tutti i valori possibili della funzione di hash, il risultato è la probabilità di azzeccare l'hash corretto che dev'essere inferiore al 100%, ossia 1)

Sapendo che $2^{20} = 1.048.576$ allora $2^{22} = 4.194.304 > 3.600.000$

quindi N = 22 è il valore minimo.

Gennaio 2022

1. Cryptography

1. Do trasposition ciphers preserve the frequency of the letters?

   ▼ Risposta

   true

2. The number of transposition ciphers applicable to a string of 4 bit is .....

   ▼ Risposta

   24

   🧠 **Spiegazione:**

   Un **cifrario a trasposizione** cambia **l'ordine** dei bit, ma **non** il loro contenuto. Quindi, dati 4 bit (es. `1010`), si possono riordinare in tutte le **permutazioni** possibili delle 4 posizioni.

   Il numero di permutazioni di 4 elementi è:

   $4! = 4 \times 3 \times 2 \times 1 = 24$

   📌 Ogni permutazione è un cifrario diverso.

3. The number of substitution ciphers applicable to a string of 4 bit is .....

   ▼ Risposta

   $2^4!$

   🧠 **Spiegazione:**

   Una stringa di **4 bit** può avere **2^4 = 16 valori diversi** (da `0000` a `1111`), ma attenzione: la **domanda chiede quanti cifrari di sostituzione** sono possibili **su una stringa di 4 bit**, non solo **sui simboli da 4 bit**.

   Se si intende **ogni possibile stringa di 4 bit**, allora ce ne sono:

   24=16(cioe` tutte le possibili combinazioni)2^4 = 16 \quad \text{(cioè tutte le possibili combinazioni)}

   24=16(cioe` tutte le possibili combinazioni)

   Un **cifrario di sostituzione** mappa **ogni possibile stringa** su un'altra stringa (senza modificarne la lunghezza), e **ogni mappatura deve essere unica** (cioè una permutazione).

   Quindi, il numero di possibili cifrari di sostituzione è:

   Numero di permutazioni di 16 valori=16!=20922789888000\text{Numero di permutazioni di 16 valori} = 16! = 20922789888000

   Numero di permutazioni di 16 valori=16!=20922789888000

Luglio 2019

What is the probability of finding a collision for an 60-bit hash function? What is the main
reason for this probability?

▼ Soluzione (con ChatGPT)

Two cases:
• if the target hash code/document is given: 2^{-60}
• if the target hash code/document can be forged by the attacker: 2^{-30} (cf. the birthday
paradox)

Spiegazione:

🔷 Caso 1: Hash target fisso (preimage attack)

> Se l'attaccante deve trovare un messaggio specifico che produce un hash noto (cioè è fissato l'hash target), allora la probabilità di indovinare correttamente è:

$\frac{1}{2^{60}} = 2^{-60}$

Dato che ci sono `2^60` possibili hash diversi, ogni tentativo ha 1 possibilità su `2^60` di indovinare esattamente quello giusto.

🔷 Caso 2: Collisione qualunque (birthday attack)

Concettualmente, nel primo caso, all'interno di un pagliaio, si cerca, avendo uno stelo in mano, il suo gemello identico (es. di lunghezza di 5 cm). Nel caso del paradosso del compleanno ci si accontenta di trovare una qualsiasi coppia di steli identica.

Se si ha un sistema con solo M possibili valori hash (ad esempio, $M = 2^N$):

1. Si prende il primo hash: non ci sono ancora collisioni.
2. Si prende il secondo hash: c'è 1 confronto da fare (tra il primo e il secondo hash).
3. Si prende il terzo hash: ci sono 2 confronti da fare (tra il terzo e il primo, e tra il terzo e il secondo).
4. SI prende il k-esimo hash: ci sono k–1 confronti da fare.

Il numero totale di coppie uniche che si possono formare da k elementi è dato dalla formula $\frac{k \cdot (k-1)}{2}$.

Il paradosso del compleanno afferma che la probabilità di trovare una collisione diventa significativa (circa 50%) quando il numero di elementi (nel nostro caso, hash generati) è circa la radice quadrata del numero totale di possibili risultati.

Matematicamente, se si hanno M possibili valori hash e si generano k hash, la probabilità che non ci siano collisioni (tutti gli hash sono unici) è approssimativamente:

P(no collisioni) ≈ $e^{-\frac{k \cdot (k-1)}{2M}}$

Si vuole che la probabilità di **avere** una collisione sia almeno il 50%, quindi P(collisione) = 1 – P(no collisioni) ≈ 0.5.

Se si risolve per k in $e^{-\frac{k \cdot (k-1)}{2M}} \approx 0.5$, si trova che k è approssimativamente $\sqrt{2M \cdot ln2}$. Poiché $ln2 \approx 0.693$, questo è circa 1,18·$\sqrt{M}$.

Nel contesto delle collisioni hash, M = 2^N, quindi il numero di hash k necessari per una probabilità del 50% di collisione è circa $\sqrt{2 \cdot 2^N ln2} = \sqrt{2^{N+1} \cdot ln2} \approx 1,18 \cdot 2^{\frac{N}{2}}$

Per esempio, se N = 128 (un hash a 128 bit), $2^{\frac{N}{2}} = 2^{64}$. Questo è un numero enorme, ma è infinitamente più piccolo di 2^{128}.

🧠 Riepilogo

```
### Probabilità di collisione con hash a 60 bit

- **Caso 1: Hash target fisso**
  Probabilità: $2^{-60}$
  → Attacco di preimage: si cerca un input che produca un hash specifico.

- **Caso 2: Collisione generica (Birthday Attack)**
  Probabilità: $2^{-30}$
  → Si cerca una qualsiasi coppia di input che producano lo stesso hash.
  → Principale causa: paradosso del compleanno.
```

Luglio 2011

1. ***Crittografia I***

Si consideri lo schema crittografico che trasforma ciascun carattere x del plaintext nel carattere $E(x)$ dato dalla seguente formula:

$$E(x) = ((a * x + b) \bmod m)$$

dove

- *l'alfabeto ha m lettere e la prima lettera dell'alfabeto è rappresentata dal numero 0, il secondo dal numero 1, ... e l'ultimo dal numero $m - 1$;*
- *a e b sono numeri interi che fungono da chiave di cifratura e a è relativamente primo con m;*
- *$(y \bmod z)$ indica il resto della divisione intera tra y e z.*

(a) Si dimostri che l'algoritmo di decifratura è dato da

$$D(x) = a^{-1}(x - b) \bmod m$$

dove a^{-1} è un inverso moltiplicativo di a, ovvero $aa^{-1} = 1 \bmod m$.

▼ Soluzione

Soluzione.

$$\begin{aligned} D(E(x)) &= a^{-1}(E(x) - b) \mod m \\ &= a^{-1}(((ax + b) \mod m) - b) \mod m \\ &= a^{-1}(ax + b - b) \mod m \\ &= a^{-1}ax \mod m \\ &= x \mod m. \end{aligned}$$

E(m) = c (cifrato)

D(c) = a^{-1}(c - b) mod M = m (messaggio in chiaro)

$$D(E(x)) = a^{-1}((a * x + b) \mod m)$$

a) D(x) = $a^{-1}(x-b) \bmod m$

= $a^{-1}((a \cdot x + b) \bmod m) - b) \bmod m = a^{-1}((ax \bmod m + b \bmod m) - b) \bmod m = a^{-1}(ax \bmod m + b \bmod m - b) \bmod m =$

= $(x \bmod m + a^{-1}b \bmod m - a^{-1}b) \bmod m = x \bmod m + a^{-1}b \bmod m - a^{-1}b \bmod m = x \bmod m$

→ viene x mod m perché ogni carattere x del plaintext viene trasformato nel carattere corrispondente appartenente all'alfabeto m (per questo fai mod m, perché deve stare all'interno di m per forza, se arriva alla fine riparte dall'inizio come ARITMETICA MODULARE DI ALGEBRA).

b. Si discuta la sicurezza dello schema crittografico.

▼ Risposta

Nonostante la maggiore complessità rispetto al cifrario di sostituzione semplice, il cifrario affine non fornisce una sicurezza elevata contro attacchi sofisticati.
Può essere vulnerabile a un attacco di crittoanalisi se l'attaccante ha accesso a una quantità sufficiente di testo cifrato e può effettuare analisi delle frequenze.

2. ***Crittografia II***

La funzione one-way utilizzata in UNIX per calcolare e memorizzare l'hash delle password degli utenti è derivata dall'algoritmo di cifratura DES modificato in modo tale che non esiste una chiave di cifratura che consenta di calcolare PW a partire dall'hash code $h(PW)$.

(a) Inoltre l'algoritmo è stato deliberatamente modificato in modo tale da essere molto più lento di DES. Perchè?

▼ Risposta

Soluzione. Per rendere più lenti (e quindi più difficili) password guessing attacks.

(b) Spesso, il file delle password invece di memorizzare coppie della forma $\langle U, h(PW_U)\rangle$ dove PW_U è la password dell'utente U, memorizza triple della forma $\langle U, R_U, h(PW_U \| R_U)\rangle$, dove R_U è un numero generato in modo (pseudo)random e $\|$ denota concatenazione. Perché?

▼ Risposta

Soluzione. Anche in questa soluzione serve a rendere più lenti (è quindi più difficili) password guessing attacks.

Luglio 2009

2. ***Crittografia***

Un sistema consente all'utente di scegliere una password con una lunghezza minima di un carattere e massima di 8 caratteri. Si assuma che sia possibile testare 10.000 password al secondo. L'amministratore di sistema vuole disabilitare le password non appena si abbia la probabilità 0.1 che sia stata scoperta.

Si determini il tempo medio dopo con cui questa probabilità viene raggiunta nel caso in cui i caratteri con cui sono formate le password siano:

(a) Caratteri ASCII con codici da 1 to 127, estremi inclusi.

Soluzione.

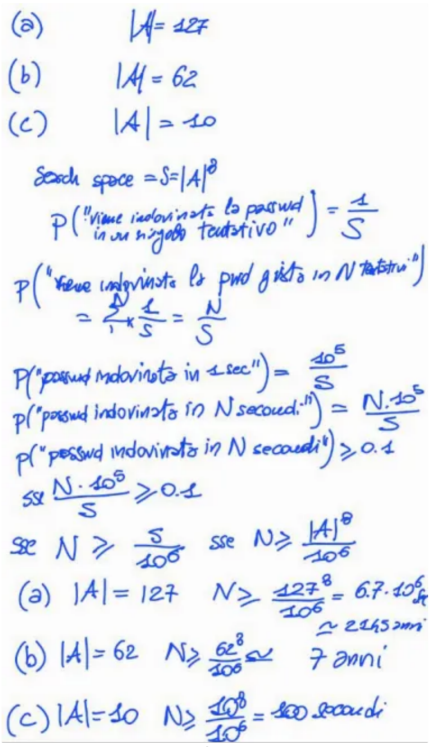
(b) Caratteri alfanumerici (da 'A' a 'Z,' da 'a' a 'z,' e da '0' a '9').

Soluzione.

(c) Numeri (da '0' a '9').

Soluzione.

▼ Soluzione



Nota: il calcolo mostrato è leggermente sbagliato. 10000 è 10^4 e non 10^5, per cui nel calcolo finale l'ordine di grandezza scala di 10:

P("indovinare in 1 sec") = $\frac{10^4}{S}$

P("indovinare in N sec") = $\frac{N \cdot 10^4}{S}$

$\frac{N \cdot 10^4}{S} \geq 0,1 \rightarrow N \geq \frac{S}{10^5}$

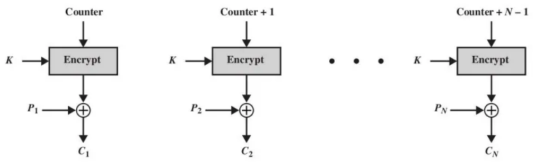
(a) |A| = 127 → $N \geq \frac{127^8}{10^5} \simeq 6,77 \cdot 10^{11}$ sec $\simeq 21454$ anni

(b) |A| = 62 → $N \geq \frac{62^8}{10^5} \simeq 2,18 \cdot 10^9$ sec $\simeq 70$ anni

(c) |A| = 10 → $N \geq \frac{10^8}{10^5} = 1000$ secondi

3. ***Crittografia***

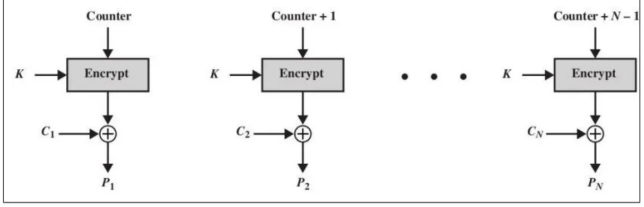
Il seguente schema crittografico per block encription. ***Counter*** *è una sequenza di bit arbitraria della stessa linghezza b dei blocchi di dati e l'operazione di somma è da intendersi modulo 2^b.*



(a) Si disegni il corrispondente schema crittografico da usarsi in fase di decifratura.

▼ Soluzione

Soluzione.



(b) Tale schema crittografico si presta a implementazioni più o meno efficienti rispetto allo schema CBC visto a lezione? Si giustifichi la risposta data.

▼ Risposta

Soluzione.
A differenza dello CBC lo schema è altamente parallelizzabile e la cifrature possono essere precomputate.

Giugno 2009

1. ***Crittografia Simmetrica e Funzioni di Hash***

Si consideri la seguente definizione di una funzione di hash basata su un algoritmo crittografico. Sia K una chiave crittografica data e sia $M = M_1 M_2 \cdots M_n$ (con $n \geq 1$) un messaggio dove gli M_i (per $1 \leq i \leq n$) sono blocchi di bit di ugual lunghezza:

$$\begin{aligned} H(M_1) &= E(K, M_1) \\ H(M_1 \cdots M_i M_{i+1}) &= E(K, H(M_1 \cdots M_i) \oplus M_{i+1}) \quad \text{per } i = 1, \ldots, n-1 \end{aligned}$$

Si dimostri che lo schema non è sicuro mostrando che dato un messaggio $A_1 A_2$ ed un blocco arbitrario B_1 è possibile determinare un blocco B_2 tale che $H(B_1 B_2) = H(A_1 A_2)$, ovvero che H non è weak collision resistant.

▼ Soluzione

$$H(A_1 A_2) = E(K, E(K, A_1) \oplus A_2)$$

mentre

$$H(B_1 B_2) = E(K, E(K, B_1) \oplus B_2)$$

Quindi è sufficiente trovare un blocco B_2 tale che

$$E(K, B_1) \oplus B_2 = E(K, A_1) \oplus A_2.$$

Mettendo in $\oplus$ ambo i lati con $E(K, B_1)$ e semplificando, otteniamo:

$$B_2 = E(K, B_1) \oplus E(K, A_1) \oplus A_2.$$

Quindi se $B_2 = E(K, B_1) \oplus E(K, A_1) \oplus A_2$, allora $H(B_1 B_2) = H(A_1 A_2)$ per qualunque valore di A_1, $A2$ e B_1. Quindi H non è *weak collision resistant*.

2. ***Crittografia***

Si consideri il seguente algoritmo crittografico basato sull'idea di sostituire ogni lettera dell'alfabeto m con $(am + b) \bmod 26$ dove la chiave è data da $k = \langle a, b \rangle$ con a e b interi positivi nell'intervallo $[0, 25]$.

Si dimostri, mediante un esempio, che se a e 26 non sono relativamente primi allora lo schema non è utilizzabile.

▼ Soluzione

$c = (am + b) \mod 26$

$a = 13, b = 0, m_1 = 2, m_2 = 4$

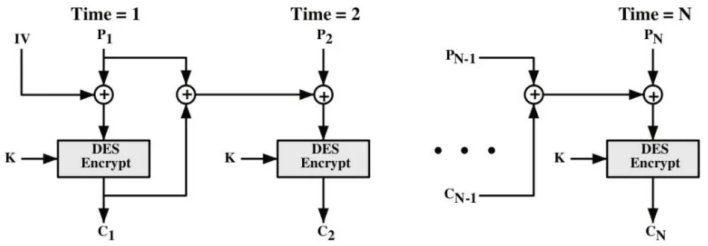
$c_1 = (13 * 2 + 0) \mod 26 = 0$

$c_2 = (13 * 4 + 0) \mod 26 = 0$

Soluzione. Si consideri la chiave $k = \langle 2, 0 \rangle$. (2 non è relativamente primo con 26.) Il ciphertext corrispondente a $m_1 = 0$ coincide con il ciphertext corrispondente a $m_2 = 13$. Ciò significa che la funzione di cifratura non è invertibile e quindi che non è sempre possibile risalire univocamente al plaintext corrispondente ad un dato ciphertext. Ciò rende lo schema proposto inutilizzabile.

3. ***Crittografia***

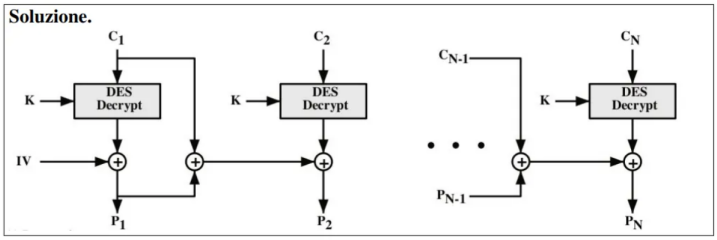
Il seguente schema crittografico per block encription *è utilizzato in Kerberos v.4.*



Si disegni il corrispondente schema crittografico da usarsi in fase di decifratura.

▼ Soluzione

Soluzione.



▼ Suggerimento in caso di difficoltà

Per risolvere questi esercizi dove viene chiesto di fornire lo schema di cifratura o decifratura, basta immaginare di posare uno specchio alla base dello schema e vederlo, così, ribaltato. L'immagine ribaltata è lo schema inverso che serve per invertire il processo. Attenzione alla direzione delle frecce che cambia anche essa.

Luglio 2008

1. ***Crittografia simmetrica***

Si consideri l'algoritmo di cifratura a blocchi definito da:

$$\begin{aligned} C_0 &= IV \\ C_i &= E_K(C_{i-1}) \oplus P_i \end{aligned} \tag{1}$$

dove IV e' il vettore di inizializzazione. Si definisca l'operazione di decifrazione.

▼ Soluzione

$$P_i = E_K(C_{i-1}) \oplus C_i$$

Infatti, se moltiplichiamo ambo il lati (1) di per $E_K(C_{i-1})$ otteniamo:

$$E_K(C_{i-1}) \oplus C_i = E_K(C_{i-1}) \oplus (E_K(C_{i-1}) \oplus P_i)$$

Sfruttando le proprietà dello $\oplus$ otteniamo:

$$E_K(C_{i-1}) \oplus C_i = P_i$$

2. ***Funzioni di Hash***

Si consideri la seguente funzione di hash. I messaggi sono visti come sequenza di numeri $M = a_1 a_2 \cdots a_l$. La funzione di hash è calcolata nel seguente modo:

$$H(M) = \left(\sum_{i=1}^{l} a_i \right) \mod n$$

per un dato valore predefinito di n. Indicare quali delle seguenti proprietà sono soddisfatte dalla funzione H. Giustificare le risposte date.

- [x] H can be applied to a block of data of any size.
- [x] H produces a fixed-length output.
  se si considera la rappresentazione binaria dei numeri
- [x] $H(x)$ is relatively easy to compute for any given x.
  il calcolo di H ha complessità lineare nella lunghezza di M
- [ ] One-way property
  se $K \in [0 \ldots n-1]$ allora il messaggio M contiene unico numero K tale che $H(M) = K$
- [ ] Weak collision resistance
  se M' è una permutazione di M, allora $H(M') = H(M)$, ma chiaramente $M' \neq M$
- [ ] Strong collision resistance

Giugno 2008

Crittografia simmetrica

Si consideri il seguente algoritmo di cifratura. Il plaintext P è suddiviso in due parti P_l e P_r. Anche la chiave K è suddivisa in due part K_l e K_r. Ovvero,

$$P = P_l|P_r \qquad K = K_l|K_r$$

Il testo cifrato C è dato dalla concatenazione di C_l e C_r, definite nel seguente modo:

$$C_r = P_r \oplus H(P_l|K_l) \quad (1)$$
$$C_l = P_l \oplus H(C_r|K_r) \quad (2)$$

dove H è una funzione di hash.

a. Quale relazione deve sussistere tra funzione di hash e le dimensioni della chiave e del
testo in chiaro affinche lo schema sia applicabile?

▶ Risposta

E' necessario che P_l ed P_r abbiano un numero di bit pari alla lunghezza del hash tornato da H.

b. Si definisca l'algoritmo che deve essere applicato per decifrare il testo cifrato C generato
con tale algoritmo assumendo che siano note la chiave K e la funzione di hash H.

▶ Soluzione

$P_l = C_l \oplus H(C_r|K_r)$
$P_r = C_r \oplus H(P_l|K_l)$ (attenzione l'ordine con cui si eseguono è importante!)

c. Mostrare che l'algoritmo è insicuro se si dispone di testi cifrati corrispondenti a testi in chiaro
che hanno la prima parte (Pl) in comune e uno dei due testi in chiaro e noto.

▼ Soluzione

I due plaintext sono della forma:

$$P = P_l|P_r$$
$$P' = P_l|P'_r$$

e dunque i ciphertexts sono ottenuti nel seguente modo:

$$C_r = P_r \oplus H(P_l|K_l) \qquad C_l = P_l \oplus H(C_r|K_r)$$
$$C'_r = P'_r \oplus H(P_l|K_l) \qquad C'_l = P_l \oplus H(C'_r|K_r)$$

P'_r può essere derivato a partire da C_r e C'_r nel seguente modo: $P'_r = (C_r \oplus C'_r) \oplus P_r$. Infatti:

$$\begin{aligned}
&(C_r \oplus C'_r) \oplus P_r \\
=&((P_r \oplus H(P_l|K_l)) \oplus (P'_r \oplus H(P_l|K_l))) \oplus P_r \\
=&((P_r \oplus P'_r) \oplus (H(P_l|K_l) \oplus H(P_l|K_l))) \oplus P_r \\
=&((P_r \oplus P'_r) \oplus \mathbf{0}) \oplus P_r \\
=&(P_r \oplus P'_r) \oplus P_r \\
=&P'_r \oplus (P_r \oplus P_r) \\
=&P'_r \oplus \mathbf{0} \\
=&P'_r
\end{aligned}$$

▼ Soluzione equivalente

Sono noti i due testi cifrati, un testo in chiaro e si sa che la prima parte dei testi in chiaro sono identiche.

Si ha: C_{l1}, C_{r1}, P_{l1}, P_{r1}, C_{l2}, C_{r2}, P_{l2}. E si sa che $P_{l1} = P_{l2}$.

Bisogna dimostrare di ottenere P_{r2}:

$P_{r2} = C_{r2} \oplus H(P_{l2}|K_l)$ (guardando la formula dal testo d'esame per P_r)

$P_{r2} = C_{r2} \oplus H(P_{l1}|K_l)$ (dato che $P_{l2} = P_{l1}$)

$H(P_{l1}|K_l)$ non è noto e bisogna determinarlo senza conoscere la chiave K, ma come? appare in altre formule?

$C_{r1} = P_{r1} \oplus H(P_{l1}|K_l) \rightarrow H(P_{l1}|K_l) = C_{r1} \oplus P_{r1}$ (Cr1 e Pr1 sono noti!!!)

Si sostituisce nella formula di P_{r2}:

$P_{r2} = C_{r2} \oplus (C_{r1} \oplus P_{r1})$ (sono tutti noti, ergo P_{r2} si può determinare)

Giugno 2007

Si assuma di avere una conoscenza parziale del seguente testo in chiaro e del corrispondente testo cifrato:
Plaintext: *UA* *IAI **ER I**E
Ciphertext: I*EC FA** OH*C KSP*
dove * indica i caratteri non noti.

Si sa che il testo cifrato e ottenuto dal testo in chiaro applicando l'algoritmo di Vigenere con lunghezza del blocco pari a 4 e utilizzando l'alfabeto inglese (ovvero ABCDEFGHIJKLMNOPQRSTUVWXYZ).
Si determini la chiave.

▼ Soluzione

0 → 25

📖 Regole Vigeneré (parti da 0 con alfabeto)

$$(Ciphertext - Key) \mod 26 = Plaintext$$

(P + K) mod 26 = C → x trovare ciphertext

(C - P) mod 26 = K → x trovare key

I → K (1) → (10 - 8) mod 26 = 2
I → A (2) → (0 - 8) mod 26 = 18
A → E (3) → (4 - 0) mod 26 = 4
R → C (4) → (2 - 17) mod 26 = 11

Key: 2 18 4 11 → C S E L

UA *IAI **ER I**E
I*EC FA** OH*C KSP*

Giugno 2006

Crittografia simmetrica

Si utilizzi la procedura di cifratura per trasposizione per codificare la sequenza di caratteri ottenuta concatenando il proprio nome e il proprio cognome ed eliminando dalla coda il minimo numero di caratteri in modo tale da ottenere una stringa di lunghezza multipla di 4.

Ad esempio, nel mio caso la stringa da considerare è:

A	l	e	s	s	a	n	d	r	o		A	r	m	a	n

Come chiave si utilizzi la permutazione 2 3 4 1.

▼ Soluzione

Nota: anche lo spazio conta come carattere (!)

Soluzione.
Plaintext: `Alessandro Arman`
Ciphertext: `lesAandso Armanr`

Innanzitutto occorre suddividere la sequenza in ingresso in blocchi di 4 caratteri ciascuno. Poi per ciascun blocco il ciphertext corrispondente è dato da:

$$E_e(m) = m_{e(1)}m_{e(2)} \cdots m_{e(t)}$$

dove nel nostro caso $e(1) = 2, e(2) = 3, e(3) = 4, e(4) = 1$.
Per il primo blocco abbiamo $E_e(\texttt{Ales}) = m_2m_3m_4m_1 = \texttt{lesA}$. Si procede analogamente per il blocchi successivi.

1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 4
Ales sand ro A rman
2 3 4 1 2 3 4 1 2 3 4 1 2 3 4 1
lesA ands o Ar manr

Luglio 2006

2.

Dire quali delle seguenti affermazioni sono FALSE, giustificando la risposta data.

Si assuma che e_1 ed e_2 sono generiche (ovvero arbitrarie) permutazioni sull'alfabeto considerato.

1. List

   [VERO/FALSO] Se $E^s_e(m)$ è una procedura di sostituzione polialfabetica, allora $E^s_{e_1}(E^s_{e_2}(m))$ è più sicura sia di $E^s_{e_1}(m)$ che di $E^s_{e_2}(m)$.

   ▼ Soluzione

   FALSO. Infatti l'applicazione consecutiva di due sostituzioni con permutazioni e1 ed e2 equivale ad una singola sostituzione con permutazione e1 ∘ e2.

2. List

   [VERO/FALSO] Se $E^t_e(m)$ è una procedura di trasposizione (transposition cipher), allora $E^t_{e_1}(E^t_{e_2}(m))$ è più sicura di $E^t_{e_1}(m)$.

   ▼ Soluzione

   FALSO. Infatti l'applicazione consecutiva di due trasposizioni con permutazioni e1 ed e2 equivale ad una singola trasposizione con permutazione e1 ∘ e2.

3. List

   [VERO/FALSO] Se $E^s_e(m)$ è una procedura di sostituzione polialfabetica e $E^t_e(m)$ è una procedura di trasposizione (transposition cipher), allora $E^s_{e_1}(E^t_{e_2}(m))$ è più sicura sia di $E^s_{e_1}(m)$ che di $E^t_{e_2}(m)$.

   ▼ Soluzione

   VERO. Ed infatti la combinazione di sostituzione e trasposizione e alla base della maggior parte degli algoritmi di cifratura simmetrica moderni.

1.

Si determini il plaintext corrispondente al ciphertext `dcbdmoet` ottenuto applicando la procedura di cifratura di Vigeneré ed utilizzando come chiave la sequenza di numeri 4 2 11.

Si consideri l'alfabeto italiano

▼ Soluzione

Dati:

- **Ciphertext**: `dcbdmoet`
- **Chiave**: 4 2 11 (ripetuta ciclicamente)
- **Alfabeto italiano**: 21 lettere

  a=0, b=1, c=2, d=3, e=4, f=5, g=6, h=7, i=8, l=9, m=10, n=11, o=12, p=13, q=14, r=15, s=16, t=17, u=18, v=19, z=20

Decifrazione passo-passo:

Pos.	Cifr.	Val.	Chiave	Calcolo	Val. Chiaro	Chiaro
1	d	3	4	(3 - 4) mod 21 = 20	20	z
2	c	2	2	(2 - 2) mod 21 = 0	0	a
3	b	1	11	(1 - 11) mod 21 = 11	11	n
4	d	3	4	(3 - 4) mod 21 = 20	20	z
5	m	10	2	(10 - 2) mod 21 = 8	8	i
6	o	12	11	(12 - 11) mod 21 = 1	1	b
7	e	4	4	(4 - 4) mod 21 = 0	0	a
8	t	17	2	(17 - 2) mod 21 = 15	15	r

Testo in chiaro:

zanzibar